

Nand2Tetris — Lab 01

Project 1: Boolean Logic (HDL Chips)

Contents

- And
- And16
- DMux
- DMux4Way
- DMux8Way
- Mux
- Mux16
- Mux4Way16
- Mux8Way16
- Not
- Not16
- Or
- Or16
- Or8Way
- Xor
- HalfAdder
- FullAdder
- Add16
- ALU

And

And.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/And.hdl
/**
 * And gate:
 * if (a and b) out = 1, else out = 0
 */
CHIP And {
    IN a, b;
    OUT out;

    PARTS:
    Nand (a=a, b=b, out=c );
    Not(in= c , out=out );
}
```

Simulation screenshot:

HDL	--LBSt	Project 1	And	Chip And	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/1/And.hdl 5 /** 6 * And gate: 7 * if (a and b) out = 1, else out = 0 8 */ 9 CHIP And { 10 IN a, b; 11 OUT out; 12 13 PARTS: 14 //// Replace this comment with your code. 15 Nand(a=a , b=b, out=c); 16 Nand(a=c , b=c , out=out); 17 }</pre>				Input pins a 0 b 0 Output pins out 0 Internal pins c 1			

And16

And16.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/And16.hdl
/**
 * 16-bit And gate:
 * for i = 0, ..., 15:
 * out[i] = a[i] And b[i]
 */
CHIP And16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    Nand(a=a[0], b=b[0], out=c);
    Not(in=c , out=out[0]);
    Nand(a=a[1], b=b[1], out=w);
    Not(in=w, out=out[1]);
    Nand(a=a[2], b=b[2], out=s);
    Not(in=s , out=out[2]);
    Nand(a=a[3], b=b[3], out=d);
    Not(in=d , out=out[3]);
    Nand(a=a[4], b=b[4], out=f);
    Not(in=f , out=out[4]);
    Nand(a=a[5], b=b[5], out=e);
    Not(in=e , out=out[5]);
    Nand(a=a[6], b=b[6], out=x);
    Not(in=x , out=out[6]);
    Nand(a=a[7], b=b[7], out=h);
    Not(in=h , out=out[7]);
    Nand(a=a[8], b=b[8], out=p);
    Not(in=p , out=out[8]);
    Nand(a=a[9], b=b[9], out=l);
    Not(in=l , out=out[9]);
    Nand(a=a[10], b=b[10], out=k);
    Not(in=k , out=out[10]);
    Nand(a=a[11], b=b[11], out=o);
    Not(in=o , out=out[11]);
    Nand(a=a[12], b=b[12], out=u);
    Not(in=u , out=out[12]);
    Nand(a=a[13], b=b[13], out=v);
    Not(in=v , out=out[13]);
    Nand(a=a[14], b=b[14], out=j);
    Not(in=j , out=out[14]);
    Nand(a=a[15], b=b[15], out=t);
    Not(in=t , out=out[15]);

}
```

Simulation screenshot:

HDL ☐ --LBSt Project 1 And16

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/And16.hdl
5 /**
6  * 16-bit And gate:
7  * for i = 0, ..., 15:
8  * out[i] = a[i] And b[i]
9  */
10 CHIP And16 {
11     IN a[16], b[16];
12     OUT out[16];
13
14     PARTS:
15     Nand16(a=a, b=b, out=n);
16     Not16(in=n, out=out);
17 }
```

Chip And16 Eval Reset Clock: 0

Input pins

a 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec

b 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec

Output pins

out 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec

Internal pins

n 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 dec

DMux

DMux.hdl

```
// This file is part of www.nand2tetrtris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux.hdl
/**
 * Demultiplexor:
 * [a, b] = [in, 0] if sel = 0
 *          [0, in] if sel = 1
 */
CHIP DMux {
    IN in, sel;
    OUT a, b;

    PARTS:
        Not(in=sel, out=w);
        And(a=in, b=w, out=a);
        And(a=in, b=sel, out=b);
}
```

Simulation screenshot:

HDL	--LBSt	Project 1	DMux	Chip DMux	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetrtris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/1/DMux.hdl 5 /** 6 * Demultiplexor: 7 * [a, b] = [in, 0] if sel = 0 8 * [0, in] if sel = 1 9 */ 10 CHIP DMux { 11 IN in, sel; 12 OUT a, b; 13 14 PARTS: 15 Not(in=sel, out=n); 16 And(a=in, b=n, out=a); 17 And(a=in, b=sel, out=b); 18 }</pre>				Input pins in 0 sel 0 Output pins a 0 b 0 Internal pins n 1			

DMux4Way

DMux4Way.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux4Way.hdl
/**
 * 4-way demultiplexor:
 * [a, b, c, d] = [in, 0, 0, 0] if sel = 00
 *                  [0, in, 0, 0] if sel = 01
 *                  [0, 0, in, 0] if sel = 10
 *                  [0, 0, 0, in] if sel = 11
 */
CHIP DMux4Way {
    IN in, sel[2];
    OUT a, b, c, d;

    PARTS:
    DMux(in=in, sel=sel[1], a=in0, b=in1);
    DMux(in=in0, sel=sel[0], a=a, b=b);
    DMux(in=in1, sel=sel[0], a=c, b=d);
}
}
```

Simulation screenshot:

HDL	--LBSt	Project 1	DMux4Way	Chip DMux4Way	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/1/DMux4Way.hdl 5 /** 6 * 4-way demultiplexor: 7 * [a, b, c, d] = [in, 0, 0, 0] if sel = 00 8 * [0, in, 0, 0] if sel = 01 9 * [0, 0, in, 0] if sel = 10 10 * [0, 0, 0, in] if sel = 11 11 */ 12 CHIP DMux4Way { 13 IN in, sel[2]; 14 OUT a, b, c, d; 15 16 PARTS: 17 DMux(in=in, sel=sel[1], a=x, b=y); 18 DMux(in=x, sel=sel[0], a=a, b=b); 19 DMux(in=y, sel=sel[0], a=c, b=d); 20 }</pre>				Input pins in 0 sel 0 0 dec			
				Output pins a 0 b 0 c 0 d 0			
				Internal pins x 0 y 0			

DMux8Way

DMux8Way.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux8Way.hdl
/**
 * 8-way demultiplexor:
 * [a, b, c, d, e, f, g, h] = [in, 0, 0, 0, 0, 0, 0, 0] if sel = 000
 *                               [0, in, 0, 0, 0, 0, 0, 0] if sel = 001
 *                               [0, 0, in, 0, 0, 0, 0, 0] if sel = 010
 *                               [0, 0, 0, in, 0, 0, 0, 0] if sel = 011
 *                               [0, 0, 0, 0, in, 0, 0, 0] if sel = 100
 *                               [0, 0, 0, 0, 0, in, 0, 0] if sel = 101
 *                               [0, 0, 0, 0, 0, 0, in, 0] if sel = 110
 *                               [0, 0, 0, 0, 0, 0, 0, in] if sel = 111
 */
CHIP DMux8Way {
    IN in, sel[3];
    OUT a, b, c, d, e, f, g, h;

    PARTS:
    DMux(in=in, sel=sel[2], a=abcd, b=efgh);
    DMux4Way(in=abcd, sel=sel[0..1], a=a, b=b, c=c, d=d);
    DMux4Way(in=efgh, sel=sel[0..1], a=e, b=f, c=g, d=h);
}
```

Simulation screenshot:

HDL

--LBSt

Project 1

DMux8Way

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/DMux8Way.hdl
5 /**
6  * 8-way demultiplexor:
7  * [a, b, c, d, e, f, g, h] = [in, 0, 0, 0, 0, 0, 0, 0]
8  *
9  *
10 *
11 *
12 *
13 *
14 *
15 */
16 CHIP DMux8Way {
17     IN in, sel[3];
18     OUT a, b, c, d, e, f, g, h;
19
20     PARTS:
21     DMux(in=in, sel=sel[2], a=x, b=y);
22     DMux(in=x, sel=sel[1], a=p, b=q);
23     DMux(in=y, sel=sel[1], a=r, b=s);
24     DMux(in=p, sel=sel[0], a=a, b=b);
25     DMux(in=q, sel=sel[0], a=c, b=d);
26     DMux(in=r, sel=sel[0], a=e, b=f);
27     DMux(in=s, sel=sel[0], a=g, b=h);
28 }
```

Chip DMux8Way

Eval Reset Clock: 0

Input pins

in 0

sel 0 0 0 dec

Output pins

a 0

b 0

c 0

d 0

e 0

f 0

g 0

h 0

Internal pins

x 0

y 0

p 0

q 0

r 0

s 0

Mux

Mux.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux.hdl
/**
 * Multiplexor:
 * if (sel = 0) out = a, else out = b
 */
CHIP Mux {
    IN a, b, sel;
    OUT out;

    PARTS:
        Not(in=sel, out=x);
        And(a=x, b=a, out=w1);
        And(a=sel, b=b, out=w2);
        Or(a=w1, b=w2, out=out);
}
```

Simulation screenshot:

HDL	--LBSt	Project 1	Mux	Chip Mux	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/1/Mux.hdl 5 /** 6 * Multiplexor: 7 * if (sel = 0) out = a, else out = b 8 */ 9 CHIP Mux { 10 IN a, b, sel; 11 OUT out; 12 13 PARTS: 14 Not(in=sel, out=x); 15 And(a=x, b=a, out=w1); 16 And(a=sel, b=b, out=w2); 17 Or(a=w1, b=w2, out=out); 18 }</pre>				Input pins			
				a	0		
				b	0		
				sel	0		
				Output pins			
				out	0		
				Internal pins			
				nSel	1		
				x	0		
				y	0		

Mux16

Mux16.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux16.hdl
/**
 * 16-bit multiplexor:
 * for i = 0, ..., 15:
 * if (sel = 0) out[i] = a[i], else out[i] = b[i]
 */
CHIP Mux16 {
    IN a[16], b[16], sel;
    OUT out[16];

    PARTS:
    Mux(a=a[0], b=b[0], sel=sel, out=out[0]);
    Mux(a=a[1], b=b[1], sel=sel, out=out[1]);
    Mux(a=a[2], b=b[2], sel=sel, out=out[2]);
    Mux(a=a[3], b=b[3], sel=sel, out=out[3]);
    Mux(a=a[4], b=b[4], sel=sel, out=out[4]);
    Mux(a=a[5], b=b[5], sel=sel, out=out[5]);
    Mux(a=a[6], b=b[6], sel=sel, out=out[6]);
    Mux(a=a[7], b=b[7], sel=sel, out=out[7]);
    Mux(a=a[8], b=b[8], sel=sel, out=out[8]);
    Mux(a=a[9], b=b[9], sel=sel, out=out[9]);
    Mux(a=a[10], b=b[10], sel=sel, out=out[10]);
    Mux(a=a[11], b=b[11], sel=sel, out=out[11]);
    Mux(a=a[12], b=b[12], sel=sel, out=out[12]);
    Mux(a=a[13], b=b[13], sel=sel, out=out[13]);
    Mux(a=a[14], b=b[14], sel=sel, out=out[14]);
    Mux(a=a[15], b=b[15], sel=sel, out=out[15]);

}
```

Simulation screenshot:

HDL

--LBst

Project 1

Mux16

1 // This file is part of www.nand2tetris.org

2 // and the book "The Elements of Computing Systems"

3 // by Nisan and Schocken, MIT Press.

4 // File name: projects/1/Mux16.hdl

5 /**

6 * 16-bit multiplexor:

7 * for i = 0, ..., 15:

8 * if (sel = 0) out[i] = a[i], else out[i] = b[i]

9 */

10 CHIP Mux16 {

11 IN a[16], b[16], sel;

12 OUT out[16];

13

14 PARTS:

15 Mux(a=a[0], b=b[0], sel=sel, out=out[0]);

16 Mux(a=a[1], b=b[1], sel=sel, out=out[1]);

17 Mux(a=a[2], b=b[2], sel=sel, out=out[2]);

18 Mux(a=a[3], b=b[3], sel=sel, out=out[3]);

19 Mux(a=a[4], b=b[4], sel=sel, out=out[4]);

20 Mux(a=a[5], b=b[5], sel=sel, out=out[5]);

21 Mux(a=a[6], b=b[6], sel=sel, out=out[6]);

22 Mux(a=a[7], b=b[7], sel=sel, out=out[7]);

23 Mux(a=a[8], b=b[8], sel=sel, out=out[8]);

24 Mux(a=a[9], b=b[9], sel=sel, out=out[9]);

25 Mux(a=a[10], b=b[10], sel=sel, out=out[10]);

26 Mux(a=a[11], b=b[11], sel=sel, out=out[11]);

27 Mux(a=a[12], b=b[12], sel=sel, out=out[12]);

28 Mux(a=a[13], b=b[13], sel=sel, out=out[13]);

29 Mux(a=a[14], b=b[14], sel=sel, out=out[14]);

30 Mux(a=a[15], b=b[15], sel=sel, out=out[15]);

31 }

Chip Mux16

Eval Reset Clock: 0

Input pins

a000000000000000dec

b000000000000000dec

sel0

Output pins

out000000000000000dec

Mux4Way16

Mux4Way16.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux4Way16.hdl
/**
 * 4-way 16-bit multiplexor:
 * out = a if sel = 00
 *      b if sel = 01
 *      c if sel = 10
 *      d if sel = 11
 */
CHIP Mux4Way16 {
    IN a[16], b[16], c[16], d[16], sel[2];
    OUT out[16];

    PARTS:
    Mux16(a=a, b=b, sel=sel[0], out=ab);
    Mux16(a=c, b=d, sel=sel[0], out=cd);
    Mux16(a=ab, b=cd, sel=sel[1], out=out);
}
```

Simulation screenshot:

HDL	--LBSt	Project 1	Mux4Way1	Chip Mux4Way16	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/1/Mux4Way16.hdl 5 /** 6 * 4-way 16-bit multiplexor: 7 * out = a if sel = 00 8 * b if sel = 01 9 * c if sel = 10 10 * d if sel = 11 11 */ 12 CHIP Mux4Way16 { 13 IN a[16], b[16], c[16], d[16], sel[2]; 14 OUT out[16]; 15 16 PARTS: 17 Mux16(a=a, b=b, sel=sel[0], out=x); 18 Mux16(a=c, b=d, sel=sel[0], out=y); 19 Mux16(a=x, b=y, sel=sel[1], out=out); 20 }</pre>				Input pins a 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec b 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec c 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec d 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec sel 0 0 dec Output pins out 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec Internal pins x 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec y 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec			

Mux8Way16

Mux8Way16.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux8Way16.hdl
/**
 * 8-way 16-bit multiplexor:
 * out = a if sel = 000
 *      b if sel = 001
 *      c if sel = 010
 *      d if sel = 011
 *      e if sel = 100
 *      f if sel = 101
 *      g if sel = 110
 *      h if sel = 111
 */
CHIP Mux8Way16 {
    IN a[16], b[16], c[16], d[16],
        e[16], f[16], g[16], h[16],
        sel[3];
    OUT out[16];

    PARTS:
    Mux4Way16(a=a, b=b, c=c, d=d, sel=sel[0..1], out=abcd);
    Mux4Way16(a=e, b=f, c=g, d=h, sel=sel[0..1], out=efgh);
    Mux16(a=abcd, b=efgh, sel=sel[2], out=out);

}
```

Simulation screenshot:

HDL● --LBStProject 1Mux8Way16

```
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/1/Mux8Way16.hdl
5 /**
6  * 8-way 16-bit multiplexor:
7  * out = a if sel = 000
8  *     b if sel = 001
9  *     c if sel = 010
10 *     d if sel = 011
11 *     e if sel = 100
12 *     f if sel = 101
13 *     g if sel = 110
14 *     h if sel = 111
15 */
16 CHIP Mux8Way16 {
17     IN a[16], b[16], c[16], d[16],
18         e[16], f[16], g[16], h[16],
19         sel[3];
20     OUT out[16];
21
22     PARTS:
23         Mux16(a=a, b=b, sel=sel[0], out=x);
24         Mux16(a=c, b=d, sel=sel[0], out=y);
25         Mux16(a=e, b=f, sel=sel[0], out=z);
26         Mux16(a=g, b=h, sel=sel[0], out=w);
27         Mux16(a=x, b=y, sel=sel[1], out=p);
28         Mux16(a=z, b=w, sel=sel[1], out=q);
29         Mux16(a=p, b=q, sel=sel[2], out=out);
30 }
```

Chip Mux8Way16EvalResetClock: 0

Input pins

a

0000000000000000

dec

b

0000000000000000

dec

c

0000000000000000

dec

d

0000000000000000

dec

e

0000000000000000

dec

f

0000000000000000

dec

g

0000000000000000

dec

h

0000000000000000

dec

sel

000

dec

Output pins

out

0000000000000000

dec

Internal pins

x

0000000000000000

dec

y

0000000000000000

dec

z

0000000000000000

dec

w

0000000000000000

dec

p

0000000000000000

dec

q

0000000000000000

dec

Not

Not.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Not.hdl
/**
 * Not gate:
 * if (in) out = 0, else out = 1
 */
CHIP Not {
    IN in;
    OUT out;

    PARTS:
        Nand(a=in , b=in, out= out);
}
```

Simulation screenshot:

HDL	--LBSt	Project 1	Not	Chip Not	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/1/Not.hdl 5 /** 6 * Not gate: 7 * if (in) out = 0, else out = 1 8 */ 9 CHIP Not { 10 IN in; 11 OUT out; 12 13 PARTS: 14 //// Replace this comment with your code. 15 Nand(a=in, b=in, out=out); 16 }</pre>				Input pins in 0			
				Output pins out 1			

Not16

Not16.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/01/Not16.hdl
/**
 * 16-bit Not gate:
 * for i = 0, ..., 15:
 *   out[i] = Not(a[i])
 */
CHIP Not16 {
    IN in[16];
    OUT out[16];

    PARTS:
    Not(in=in[0] ,out=out[0] );
    Not(in=in[1], out=out[1]);
    Not(in=in[2], out=out[2]);
    Not(in=in[3], out=out[3]);
    Not(in=in[4], out=out[4]);
    Not(in=in[5], out=out[5]);
    Not(in=in[6], out=out[6]);
    Not(in=in[7], out=out[7]);
    Not(in=in[8], out=out[8]);
    Not(in=in[9], out=out[9]);
    Not(in=in[10], out=out[10]);
    Not(in=in[11], out=out[11]);
    Not(in=in[12], out=out[12]);
    Not(in=in[13], out=out[13]);
    Not(in=in[14], out=out[14]);
    Not(in=in[15], out=out[15]);
}
```

Simulation screenshot:

The screenshot displays the HDL simulation environment. On the left, the HDL code for the Not16 chip is shown in a dark-themed editor. The code defines a 16-bit Not gate, where each output bit is the logical NOT of the corresponding input bit. On the right, the 'Chip Not16' panel shows the current state of the chip. The 'Input pins' section shows the input vector 'in' as 0000000000000000. The 'Output pins' section shows the output vector 'out' as 1111111111111111, indicating that all input bits are inverted. The top of the interface includes a toolbar with 'HDL', '--LBSt', 'Project 1', and 'Not16' dropdowns, along with 'Eval', 'Reset', and 'Clock: 0' buttons.

Section	Value
Input pins	in: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Output pins	out: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Or

Or.hdl

```
// This file is part of www.nand2tetrtris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or.hdl
/**
 * Or gate:
 * if (a or b) out = 1, else out = 0
 */
CHIP Or {
    IN a, b;
    OUT out;

    PARTS:
    Nand(a=a , b=a, out=x);
    Nand(a=b, b=b, out=y );
    Nand(a=x, b=y , out=out);
}
```

Simulation screenshot:

HDL	--LBSt	Project 1	Or	Chip Or	Eval	Reset	Clock: 0
1				Input pins			
2				a	0		
3				b	0		
4				Output pins			
5				out	0		
6				Internal pins			
7				nota	1		
8				notb	1		
9				andout	1		
10							
11							
12							
13							
14							
15							
16							
17							
18							
19							

Or16

Or16.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or16.hdl
/**
 * 16-bit Or gate:
 * for i = 0, ..., 15:
 * out[i] = a[i] Or b[i]
 */
CHIP Or16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    Or(a=a[0] , b=b[0] , out=out[0] );
    Or(a=a[1] , b=b[1] , out=out[1] );
    Or(a=a[2] , b=b[2] , out=out[2] );
    Or(a=a[3] , b=b[3] , out=out[3] );
    Or(a=a[4] , b=b[4] , out=out[4] );
    Or(a=a[5] , b=b[5] , out=out[5] );
    Or(a=a[6] , b=b[6] , out=out[6] );
    Or(a=a[7] , b=b[7] , out=out[7] );
    Or(a=a[8] , b=b[8] , out=out[8] );
    Or(a=a[9] , b=b[9] , out=out[9] );
    Or(a=a[10] , b=b[10] , out=out[10] );
    Or(a=a[11] , b=b[11] , out=out[11] );
    Or(a=a[12] , b=b[12] , out=out[12] );
    Or(a=a[13] , b=b[13] , out=out[13] );
    Or(a=a[14] , b=b[14] , out=out[14] );
    Or(a=a[15] , b=b[15] , out=out[15] );

}
```

Simulation screenshot:

The screenshot displays the HDL simulation environment. On the left, the HDL code for the Or16 chip is shown, with line numbers 1 through 18. The code implements a 16-bit OR gate using a series of Or16 parts. On the right, the 'Chip Or16' panel shows the current state of the chip's pins. The input pins 'a' and 'b' are both set to 0. The output pin 'out' is also 0. The internal pins 'na' and 'nb' are both set to 1. The 'Eval' button is highlighted, indicating the current state of the simulation.

Pin	Value
a	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
b	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
out	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
na	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
nb	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Or8Way

Or8Way.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or8Way.hdl
/**
 * 8-way Or gate:
 * out = in[0] Or in[1] Or ... Or in[7]
 */
CHIP Or8Way {
    IN in[8];
    OUT out;

    PARTS:
    Or(a=in[0], b=in[1], out=t1);
    Or(a=t1, b=in[2], out=t2);
    Or(a=t2, b=in[3], out=t3);
    Or(a=t3, b=in[4], out=t4);
    Or(a=t4, b=in[5], out=t5);
    Or(a=t5, b=in[6], out=t6);
    Or(a=t6, b=in[7], out=out);

}
```

Simulation screenshot:

HDL	--LBSt	Project 1	Or8Way	Chip Or8Way	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/1/Or8Way.hdl 5 /** 6 * 8-way Or gate: 7 * out = in[0] Or in[1] Or ... Or in[7] 8 */ 9 CHIP Or8Way { 10 IN in[8]; 11 OUT out; 12 13 PARTS: 14 Or(a=in[0], b=in[1], out=x); 15 Or(a=x, b=in[2], out=y); 16 Or(a=y, b=in[3], out=z); 17 Or(a=z, b=in[4], out=w); 18 Or(a=w, b=in[5], out=p); 19 Or(a=p, b=in[6], out=q); 20 Or(a=q, b=in[7], out=out); 21 }</pre>				Input pins in 0 0 0 0 0 0 0 0 dec			
				Output pins out 0			
				Internal pins x 0 y 0 z 0 w 0 p 0 q 0			

Xor

Xor.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Xor.hdl
/**
 * Exclusive-or gate:
 * if ((a and Not(b)) or (Not(a) and b)) out = 1, else out = 0
 */
CHIP Xor {
    IN a, b;
    OUT out;

    PARTS:
        Nand(a=a , b=b, out=w );
        Nand(a=a , b=w , out=v );
        Nand(a=b , b=w , out=x );
        Nand(a=v, b=x , out=out );
}
```

Simulation screenshot:

HDL	--LBSt	Project 1	Xor	Chip Xor	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/1/Xor.hdl 5 /** 6 * Exclusive-or gate: 7 * if ((a and Not(b)) or (Not(a) and b)) out = 1, els 8 */ 9 CHIP Xor { 10 IN a, b; 11 OUT out; 12 13 PARTS: 14 Not(in=a, out=na); 15 Not(in=b, out=nb); 16 And(a=a, b=nb, out=x); 17 And(a=na, b=b, out=y); 18 Or(a=x, b=y, out=out); 19 }</pre>				Input pins a 0 b 0 Output pins out 0 Internal pins na 1 nb 1 x 0 y 0			

HalfAdder

HalfAdder.hdl

```
CHIP HalfAdder {  
    IN a, b;  
    OUT sum, carry;  
  
    PARTS:  
    Xor(a=a, b=b, out=sum);  
    And(a=a, b=b, out=carry);  
}
```

Simulation screenshot:

HDL	--LBSt	Project 2	HalfAdder	Chip HalfAdder	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/2/HalfAdder.hdl 5 /** 6 * Computes the sum of two bits. 7 */ 8 CHIP HalfAdder { 9 IN a, b; // 1-bit inputs 10 OUT sum, // Right bit of a + b 11 carry; // Left bit of a + b 12 13 PARTS: 14 Xor(a=a, b=b, out=sum); 15 And(a=a, b=b, out=carry); 16 }</pre>				Input pins a 1 b 1 Output pins sum 0 carry 1			

FullAdder

FullAdder.hdl

```
CHIP FullAdder {
    IN a, b, c;
    OUT sum, carry;

    PARTS:
    HalfAdder(a=a, b=b, sum=sum1, carry=carry1);
    HalfAdder(a=sum1, b=c, sum=sum, carry=carry2);
    Or(a=carry1, b=carry2, out=carry);
}
```

Simulation screenshot:

HDL	--LBSt	Project 2	FullAdder	Chip FullAdder	Eval	Reset	Clock: 0
<pre>1 // This file is part of www.nand2tetris.org 2 // and the book "The Elements of Computing Systems" 3 // by Nisan and Schocken, MIT Press. 4 // File name: projects/2/FullAdder.hdl 5 /** 6 * Computes the sum of three bits. 7 */ 8 CHIP FullAdder { 9 IN a, b, c; // 1-bit inputs 10 OUT sum, // Right bit of a + b + c 11 carry; // Left bit of a + b + c 12 13 PARTS: 14 HalfAdder(a=a, b=b, sum=sum1, carry=carry1); 15 HalfAdder(a=sum1, b=c, sum=sum, carry=carry2); 16 Or(a=carry1, b=carry2, out=carry); 17 }</pre>				Input pins			
				a	1		
				b	1		
				c	1		
				Output pins			
				sum	1		
				carry	1		
				Internal pins			
				sum1	0		
				carry1	1		
				carry2	0		

Add16

Add16.hdl

```
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:

        // Bit 0: Half Adder
        HalfAdder(
            a=a[0],
            b=b[0],
            sum=out[0],
            carry=c1
        );

        // Bits 1-15: Full Adders
        FullAdder(
            a=a[1],
            b=b[1],
            c=c1,
            sum=out[1],
            carry=c2
        );

        FullAdder(
            a=a[2],
            b=b[2],
            c=c2,
            sum=out[2],
            carry=c3
        );

        FullAdder(
            a=a[3],
            b=b[3],
            c=c3,
            sum=out[3],
            carry=c4
        );

        FullAdder(
            a=a[4],
            b=b[4],
            c=c4,
            sum=out[4],
            carry=c5
        );

        FullAdder(
            a=a[5],
            b=b[5],
            c=c5,
            sum=out[5],
```

```
        carry=c6
    );

    FullAdder(
        a=a[6],
        b=b[6],
        c=c6,
        sum=out[6],
        carry=c7
    );

    FullAdder(
        a=a[7],
        b=b[7],
        c=c7,
        sum=out[7],
        carry=c8
    );

    FullAdder(
        a=a[8],
        b=b[8],
        c=c8,
        sum=out[8],
        carry=c9
    );

    FullAdder(
        a=a[9],
        b=b[9],
        c=c9,
        sum=out[9],
        carry=c10
    );

    FullAdder(
        a=a[10],
        b=b[10],
        c=c10,
        sum=out[10],
        carry=c11
    );

    FullAdder(
        a=a[11],
        b=b[11],
        c=c11,
        sum=out[11],
        carry=c12
    );

    FullAdder(
        a=a[12],
        b=b[12],
        c=c12,
        sum=out[12],
```

```

        carry=c13
    );

    FullAdder(
        a=a[13],
        b=b[13],
        c=c13,
        sum=out[13],
        carry=c14
    );

    FullAdder(
        a=a[14],
        b=b[14],
        c=c14,
        sum=out[14],
        carry=c15
    );

    FullAdder(
        a=a[15],
        b=b[15],
        c=c15,
        sum=out[15],
        carry=c16
    );
}

```

Simulation screenshot:

HDL
--LBSt
Project 2
Add16

```

1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/2/Add16.hdl
5 /**
6  * 16-bit adder: Adds two 16-bit two's complement values.
7  * The most significant carry bit is ignored.
8  */
9 CHIP Add16 {
10     IN a[16], b[16];
11     OUT out[16];
12
13     PARTS:
14     FullAdder(a=a[0], b=b[0], c=false, sum=out[0], carry=c1);
15     FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
16     FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
17     FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
18     FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
19     FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c6);
20     FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c7);
21     FullAdder(a=a[7], b=b[7], c=c7, sum=out[7], carry=c8);
22     FullAdder(a=a[8], b=b[8], c=c8, sum=out[8], carry=c9);
23     FullAdder(a=a[9], b=b[9], c=c9, sum=out[9], carry=c10);
24     FullAdder(a=a[10], b=b[10], c=c10, sum=out[10], carry=c11);
25     FullAdder(a=a[11], b=b[11], c=c11, sum=out[11], carry=c12);
26     FullAdder(a=a[12], b=b[12], c=c12, sum=out[12], carry=c13);
27     FullAdder(a=a[13], b=b[13], c=c13, sum=out[13], carry=c14);
28     FullAdder(a=a[14], b=b[14], c=c14, sum=out[14], carry=c15);
29     FullAdder(a=a[15], b=b[15], c=c15, sum=out[15], carry=ignore);
30 }

```

Chip Add16
Eval Reset Clock: 0

Input pins

a 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 dec

b 1 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0 dec

Output pins

out 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 dec

Internal pins

c1 0

c2 0

c3 1

c4 0

c5 1

c6 1

c7 1

c8 0

c9 0

c10 0

c11 0

c12 0

c13 1

c14 0

c15 0

ignore 0

ALU

ALU.hdl

```
/**
 * Hack ALU
 */

CHIP ALU {
    IN
        x[16], y[16],
        zx, nx, zy, ny, f, no;

    OUT
        out[16],
        zr,
        ng;

    PARTS:

        // x preprocessing
        Mux16(a=x, b=false, sel=zx, out=xZeroed);
        Not16(in=xZeroed, out=xNot);
        Mux16(a=xZeroed, b=xNot, sel=nx, out=xProcessed);

        // y preprocessing
        Mux16(a=y, b=false, sel=zy, out=yZeroed);
        Not16(in=yZeroed, out=yNot);
        Mux16(a=yZeroed, b=yNot, sel=ny, out=yProcessed);

        // AND or ADD
        And16(a=xProcessed, b=yProcessed, out=andResult);
        Add16(a=xProcessed, b=yProcessed, out=addResult);

        Mux16(
            a=andResult,
            b=addResult,
            sel=f,
            out=preOut
        );

        // Final negation
        Not16(in=preOut, out=outNot);

        Mux16(
            a=preOut,
            b=outNot,
            sel=no,
            out=out
        );

        // zr = 1 when out == 0
        Or8Way(in=out[0..7], out=orLow);
        Or8Way(in=out[8..15], out=orHigh);
        Or(a=orLow, b=orHigh, out=orAll);
        Not(in=orAll, out=zr);
    }
}
```

```

// ng = MSB of output
And(a=out[15], b=true, out=ng);
}

```

Simulation screenshot:

The screenshot displays a Verilog simulation environment. On the left, the HDL code for a 16-bit ALU is shown, implementing various operations like addition, subtraction, and bitwise logic. On the right, the 'Chip ALU' interface is visible, showing input pins (x, y, zx, nx, zy, ny, f, no), output pins (out, zr, ng), and internal pins (xl, rxl, x2, yl, nyl, y2, fAnd, fAdd, fOut, nOut, low8, high8, orLow, orHigh, notZero). The interface includes a 'Eval' button and a 'Clock' counter.

HDL Code:

```

10 * x + 1, y + 1, x - 1, y - 1.
11 * x + y, x - y, y - x.
12 * x & y, x | y.
13 * on the 16-bit inputs x, y,
14 * according to the input bits zx, nx, zy, ny, f, no.
15 * In addition, computes the two output bits:
16 * if (out == 0) zr = 1, else zr = 0
17 * if (out < 0) ng = 1, else ng = 0
18 */
19 // Implementation: Manipulates the x and y inputs
20 // and operates on the resulting values, as follows:
21 // if (zx == 1) sets x = 0 // 16-bit constant
22 // if (nx == 1) sets x = !x // bitwise not
23 // if (zy == 1) sets y = 0 // 16-bit constant
24 // if (ny == 1) sets y = !y // bitwise not
25 // if (f == 1) sets out = x + y // Integer 2's complement addition
26 // if (f == 0) sets out = x & y // bitwise and
27 // if (no == 1) sets out = !out // bitwise not
28
29 CHIP ALU {
30   in
31   x[16], y[16], // 16-bit inputs
32   zx, // zero the x input?
33   nx, // negate the x input?
34   zy, // zero the y input?
35   ny, // negate the y input?
36   f, // compute (out = x + y) or (out = x & y)?
37   no; // negate the out output?
38   out
39   out[16], // 16-bit output
40   zr, // if (out == 0) equals 1, else 0
41   ng; // if (out < 0) equals 1, else 0
42
43   PARTS:
44   // Step 1: Handle zx and nx (zero/negate x)
45   Mux16(a=0, b=false, sel=zx, out=x1);
46   Not16(in=x1, out=nx1);
47   Mux16(a=x1, b=nx1, sel=nx, out=x2);
48
49   // Step 2: Handle zy and ny (zero/negate y)
50   Mux16(a=0, b=false, sel=zy, out=y1);
51   Not16(in=y1, out=ny1);
52   Mux16(a=y1, b=ny1, sel=ny, out=y2);
53
54   // Step 3: f = AND or ADD
55   And16(a=x2, b=y2, out=fAnd);
56   Add16(a=x2, b=y2, out=fAdd);
57   Mux16(a=fAnd, b=fAdd, sel=f, out=fOut);
58
59   // Step 4
60   Not16(in=fOut, out=nfOut);
61   Mux16(a=fOut, b=nfOut, sel=no, out=out, out[8..7]=low8, out[8..15]=high8, out[15]=ng);
62
63   // Step 5
64   Or8Way(in=low8, out=orLow);
65   Or8Way(in=high8, out=orHigh);
66   Or(a=orLow, b=orHigh, out=notZero);
67   Not(in=notZero, out=zr);
68
69 }

```

Chip ALU Interface:

- Input pins:** x (0000000000000000), y (0000000000000000), zx (0), nx (0), zy (0), ny (0), f (0), no (0).
- Output pins:** out (0000000000000000), zr (1), ng (0).
- Internal pins:** xl (0000000000000000), rxl (1111111111111111), x2 (0000000000000000), yl (0000000000000000), nyl (1111111111111111), y2 (0000000000000000), fAnd (0000000000000000), fAdd (0000000000000000), fOut (0000000000000000), nOut (1111111111111111), low8 (00000000), high8 (00000000), orLow (0), orHigh (0), notZero (0).